

Rapport de projet de Digital Desktop Prototyping

Projet académique — Année 2025–2026

PLATEFORME DESKTOP D'EXPLICABILITÉ DES MODÈLES IA

XAI Studio

ML Core · XAI · Agent

Architecture en trois couches · Tkinter avancé · Tool-calling LLM dynamique

Réalisé par

AMLLAL Amine
ZOUGA Mouhcine
AKEBLI Fatima-Ezzahrae

Encadré par

M. Brahim Bakkas

Table des matières

1	Introduction et proposition de valeur	2
1.1	Une plateforme desktop, Tkinter poussé à son maximum	2
1.2	Anatomie technique de l'interface Tkinter	3
1.3	Visite guidée de l'interface	4
2	Architecture et ingénierie backend	10
2.1	Vue d'ensemble en trois couches	10
2.2	Agent (Copilot) : raisonnement et flux de décision	11
2.3	Tools et flux d'exécution	12
2.4	Mémoire : contexte et persistance	14
2.5	Providers IA : Groq et Gemini	14
3	Évolution du projet et discipline d'ingénierie	16
3.1	Phase 1 — Fondations architecturales	16
3.2	Phase 2 — Maturation UI et premier élan ML	16
3.3	Phase 3 — Industrialisation du module de prédiction	17
3.4	Phase 4 — L'agent IA comme couche d'orchestration	18
3.5	Phase 5 — Consolidation finale	18
3.6	Synthèse et conclusion	19

1

Introduction et proposition de valeur

1.1 Une plateforme desktop, Tkinter poussé à son maximum

XAI Studio démontre qu’avec une ingénierie rigoureuse, la bibliothèque standard Tkinter de Python permet de livrer une expérience desktop comparable à celle des suites professionnelles modernes — sans *webview*, sans *framework* lourd, sans dépendance native.

XAI Studio est une plateforme desktop développée en Python autour de Tkinter et de son extension `ttk`. Elle couvre la totalité du cycle de vie d’un projet de *Machine Learning* — ingestion CSV, préprocessing, entraînement multi-modèles, évaluation comparative, inférence, explicabilité (SHAP, LIME, PDP) et détection de biais — le tout pilotable manuellement *ou* par un agent IA persistant connecté à deux *providers* LLM cloud.

Notre proposition de valeur tient en trois axes d’ingénierie :

Maturité architecturale

Une séparation stricte en trois couches (`ui` / `services` / `core`) qui isole la présentation, l’orchestration métier et la logique ML pure. Aucune importation croisée indésirable : chaque couche ne dépend que de la suivante, ce qui garantit la testabilité et la maintenabilité.

Tkinter porté à un niveau professionnel

Là où Tkinter est souvent réduit à une bibliothèque de prototypage rapide, nous l’avons exploité à fond : *theming* dynamique `ttk.Style`, *scaling* HiDPI automatique, conteneurs scrollables sur `tk.Canvas`, gestion fine des événements (`<Configure>`, `<MouseWheel>`, `<Button-1>`), *threading* cadencé par `widget.after()` pour ne jamais bloquer la boucle principale.

Agent IA comme couche d’orchestration

L’application intègre un copilote LLM persistant avec *tool-calling* natif, capable de manipuler

le *pipeline* visuel et d'exécuter des actions réelles côté **services/**, sans court-circuiter les couches sous-jacentes.

Synthèse fonctionnelle

- **Chargement de données** CSV avec détection automatique d'encodage et de séparateur.
- **Préprocessing** paramétrable (imputation, encodage, *scaling*, *split*, gestion du schéma).
- **Entraînement multi-modèles** : 6 classifieurs et 6 régresseurs *scikit-learn*.
- **Évaluation** : tableau comparatif des performances avec métriques adaptées à la tâche.
- **Prédiction** avec support des catégories et transformations cohérentes en inférence.
- **XAI** : SHAP, LIME, PDP et tableaux de bord locaux/globaux.
- **Détection de biais** via une vue dédiée à l'équité.
- **Agent IA** persistant (Groq + Gemini) avec outils dynamiques.
- **UI professionnelle** `ttk`, thèmes *light/dark* et i18n FR/EN.

1.2 Anatomie technique de l'interface Tkinter

Avant d'examiner les vues une à une, fixons le vocabulaire et les choix structurels qui sous-tendent toute l'UI.

Fondations Tkinter de l'application

- **Racine & mise à l'échelle** — la racine `tk.Tk()` est instanciée par `XAIStudioApp`, puis passée dans un détecteur de DPI maison (`detect_ui_scale`) qui propage le facteur via `root.tk.call('tk', 'scaling', ...)`. Toutes les dimensions sont ensuite multipliées par ce facteur, ce qui donne un rendu net sur écran 4K comme sur HD.
- **Composition par Frame** — la fenêtre principale est partitionnée en *TopBar* + *Sidebar* + *Content* + *StatusBar* via un assemblage hiérarchique de `tk.Frame` et `ttk.Frame`. Le code recense plus de 230 `Frame` et 177 `Label`, témoignant d'une décomposition fine en composants.
- **Theming via `ttk.Style`** — un module `ui/theme.py` construit dynamiquement deux thèmes (*light / dark*) en configurant les styles `TFrame`, `TLabel`, `TButton`, `Treeview` et `TNotebook`. Le *toggle* déclenche un *re-styling* instantané sans rechargement de la fenêtre.
- **Réactivité d'état** — 43 `StringVar` et 12 `BooleanVar` servent de liaison réactive entre *wid-gets* et modèle de données. Les changements déclenchent automatiquement les `trace_add()` observateurs sans nécessiter de *redraw* manuel.
- **Boucle UI non bloquante** — toutes les opérations coûteuses (lecture CSV, entraînement, appels LLM) sont déportées dans des `threading.Thread` (10 occurrences) et leurs résultats sont réinjectés dans le *mainloop* via `root.after()` (42 occurrences) pour respecter le contrat mono-thread de Tk.

1.3 Visite guidée de l'interface

Les sous-sections suivantes parcourent les vues principales dans l'ordre du *workflow* naturel. Pour chacune, nous décrivons l'ergonomie, la composition Tkinter sous-jacente et les optimisations visuelles mises en œuvre.

1.3.1 Vue « Données » — ingestion et exploration

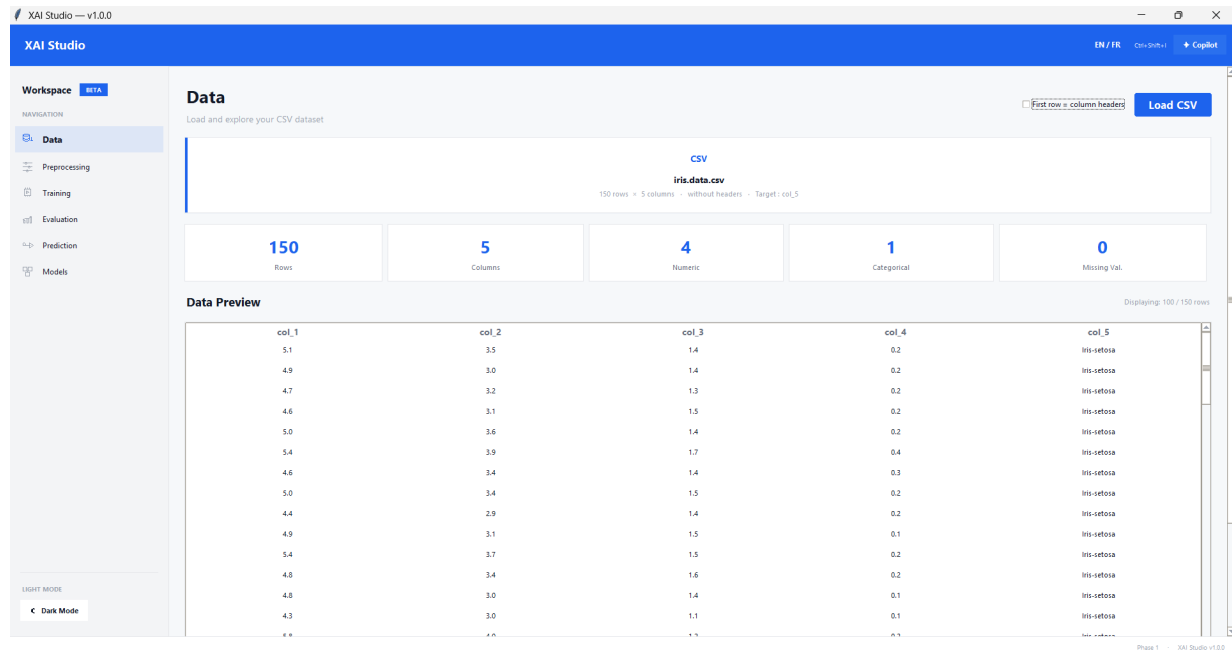


Figure 1.1 — Écran d'ingestion : aperçu tabulaire (*ttk.Treeview*), statistiques descriptives et sélection de la cible.

L'utilisateur découvre ici la *DataView*, une `ttk.Frame` structurée en deux colonnes par `grid()` : à gauche un `ttk.Treeview` configuré en mode tabulaire (colonnes dynamiques déduites du *DataFrame* pandas), à droite un panneau `ttk.Frame` de métadonnées. Le *Treeview* est enveloppé dans un `tk.Canvas` + `ttk.Scrollbar` bidirectionnels, technique classique de Tkinter pour offrir un *scroll* virtuel performant même sur de très grands datasets. Les statistiques (taille, taux de valeurs manquantes, type inféré par colonne) sont rendues sous forme de « *metric tiles* » — des `tk.Frame` arrondis factorisés en composant réutilisable. La sélection de la cible est matérialisée par une `ttk.Combobox` liée à une `StringVar` dont le `trace_add('write')` déclenche immédiatement la propagation à `PipelineService`.

1.3.2 Vue « Préprocessing » — pipeline visuel

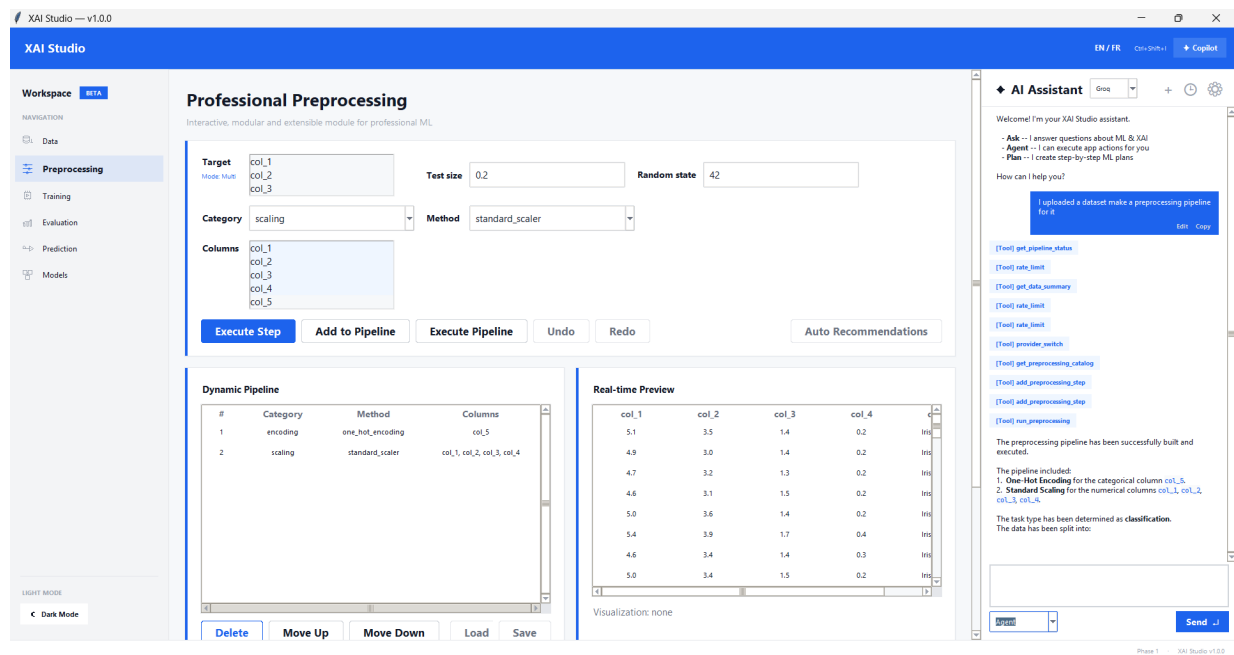


Figure 1.2 — Studio de préprocessing : composition d'étapes (imputation, encodage, scaling, split) avec aperçu en temps réel.

La `PreprocessingView` est l'un des points où Tkinter est le plus sollicité. Chaque étape du *pipeline* est rendue comme une « carte » (un `tk.Frame` stylisé avec bordure, *padding* et *badge* de statut) instanciée dynamiquement à la volée. La liste verticale est placée dans un `tk.Canvas` dont la *scrollregion* est recalculée à chaque `<Configure>` pour s'adapter au nombre variable d'étapes. Les contrôles par étape combinent `ttk.Combobox`, `ttk.Spinbox` et `ttk.Checkbutton`, tous reliés à des variables Tk pour un binding réactif. Une particularité remarquable : cette même vue est manipulable par l'agent IA via l'outil `add_preprocessing_step`, ce qui implique qu'*aucun état n'est piégé dans les widgets* — la source de vérité reste le `PipelineService`, et l'UI se contente de le refléter.

1.3.3 Vue « Entraînement » — multi-modèles asynchrone

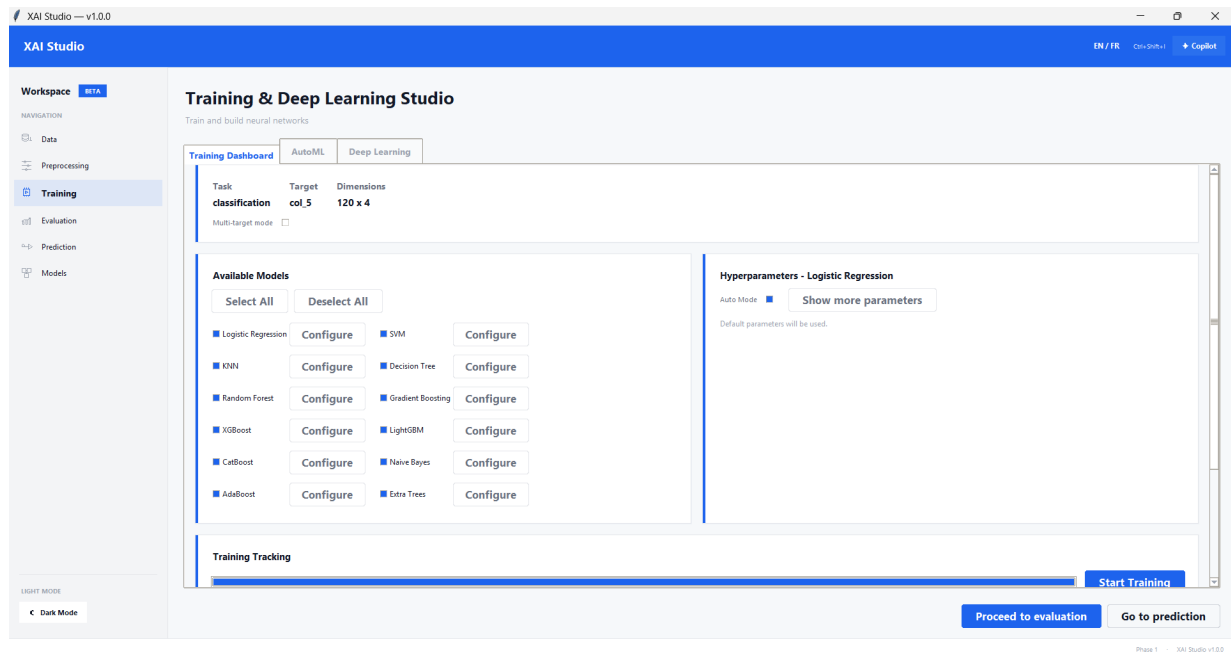


Figure 1.3 – Studio d’entraînement : sélection des modèles, hyper-paramètres et suivi de progression en direct.

La `TrainingView` illustre le pattern « UI réactive sur calcul lourd ». L’utilisateur sélectionne ses modèles via une matrice de `ttk.Checkbutton` reliée à des `BooleanVar`. Au lancement, l’entraînement est confié à un `threading.Thread` démon; chaque *tick* de progression est ensuite repoussé dans le *mainloop* via `root.after(0, callback)`, ce qui préserve la règle d’or de Tk (un seul *thread* accroché à la boucle d’événements). Une `ttk.Progressbar` en mode *determinate* affiche l’avancement, doublée d’une `tk.Label` ETA. Les hyper-paramètres sont éditables via des `ttk.Spinbox` et `ttk.Entry` validés à la sortie de focus (`<FocusOut>`).

1.3.4 Vue « Évaluation » — comparaison synthétique

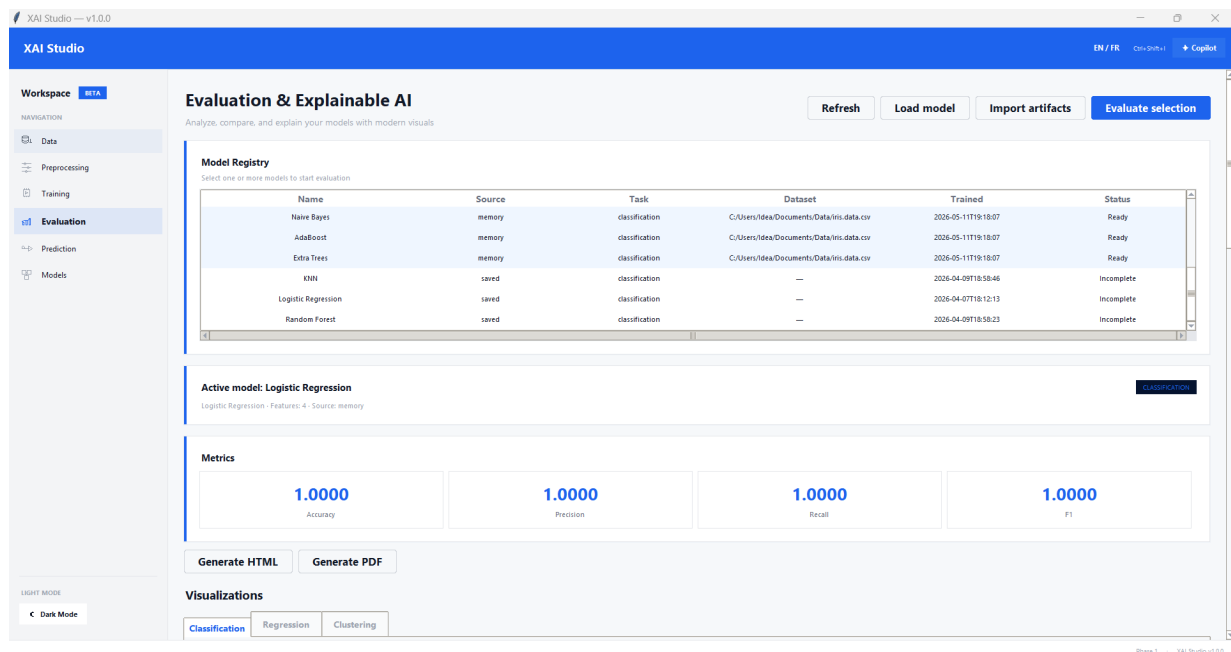


Figure 1.4 — Tableau comparatif des modèles entraînés avec métriques adaptées à la tâche (classification/régression).

La vue d'évaluation présente un `ttk.Treeview` multi-colonnes configuré par `column(..., anchor='center', stretch=True)`. Les en-têtes sont triables : un `bind('<Button-1>', sort_handler)` sur chaque *heading* déclenche un tri stable en place. Les métriques (*accuracy*, F1, RMSE, R^2 ...) sont sélectionnées automatiquement selon le type de tâche détecté dans `PipelineService.evaluation_kind`. Un *tag* de couleur souligne la ligne du meilleur modèle, appliqué via `tree.tag_configure('best', background=...)`.

1.3.5 Vue « Modèles » — catalogue persisté

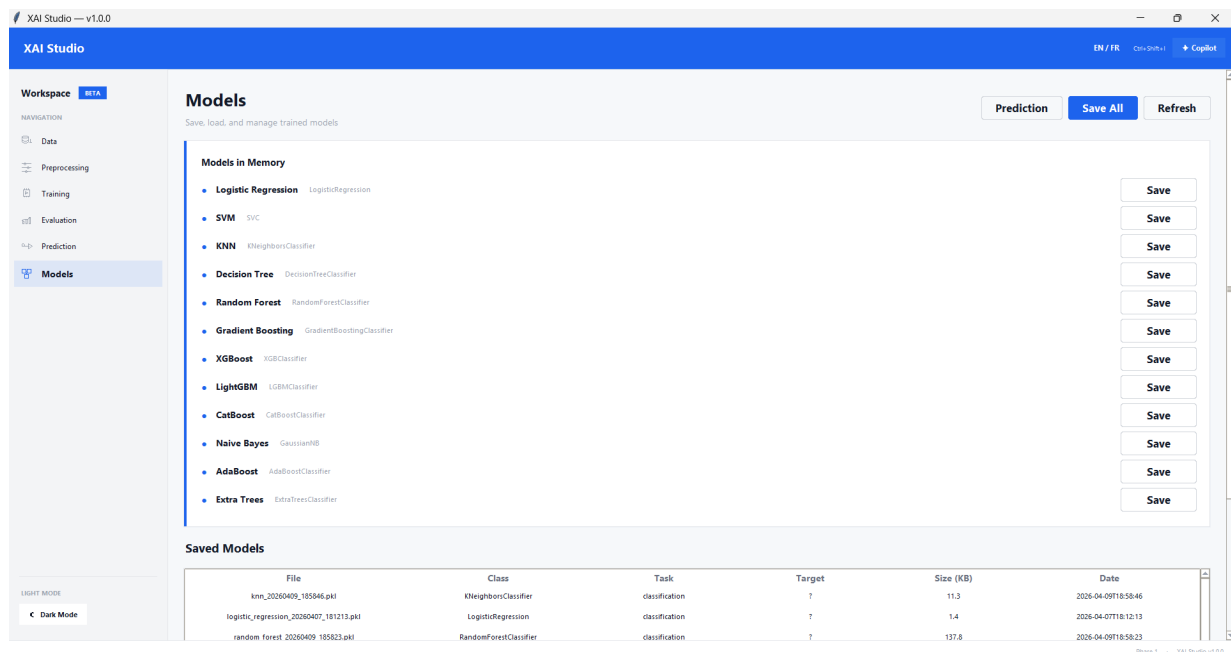


Figure 1.5 — Catalogue des modèles sauvegardés : métadonnées, schéma d’entrée et navigation rapide vers l’inférence.

La `ModelsView` expose les artefacts `.pkl` présents dans `models/saved/`. Chaque modèle apparaît sous forme de « fiche » — un `tk.Frame` arrondi (*rounded rectangle* dessiné via `tk.Canvas.create_polygon` avec les coins en `smooth=True`) qui énumère les métadonnées chargées à la volée : nom, algorithme, métriques clés, schéma d’entrée. Les boutons d’action utilisent des `tk.Label clickable` (plutôt que `ttk.Button`) afin de contrôler très finement la typographie et les états hover via `bind('<Enter>')` / `bind('<Leave>')` — un détour classique mais payant pour atteindre un rendu de niveau produit.

1.3.6 Vue « Prédiction » — inférence et what-if

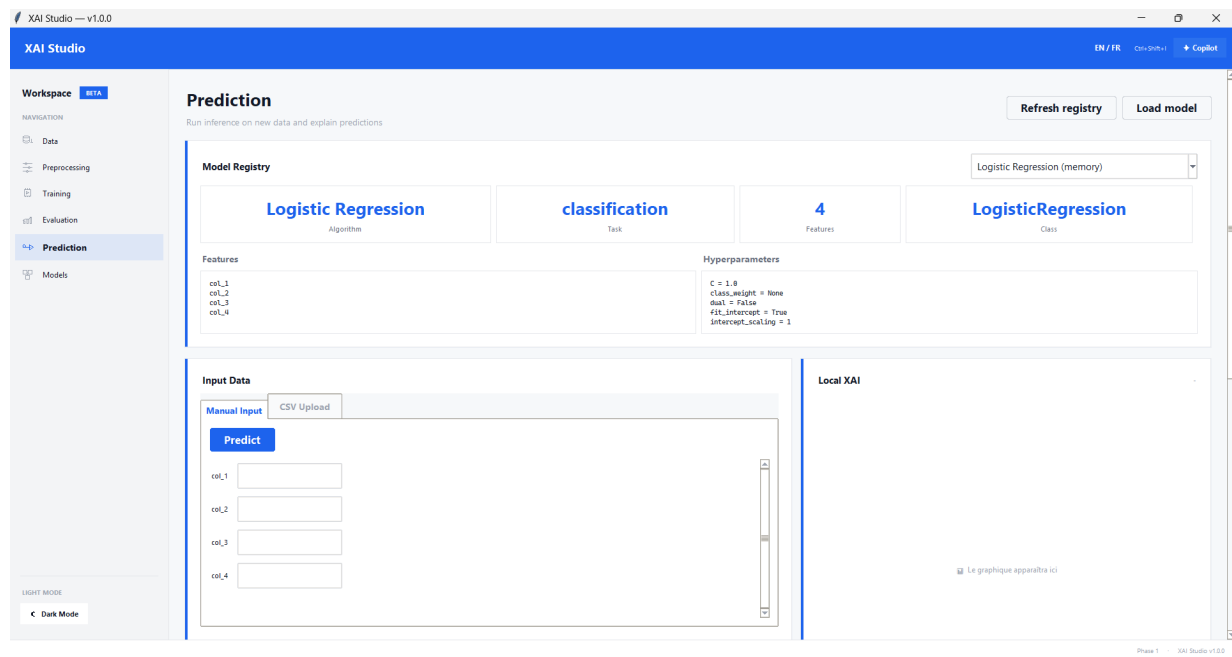


Figure 1.6 – Module d’inférence : saisie manuelle, import CSV par lot, analyse what-if et explications SHAP locales.

La `PredictionView` démontre l’élégance qu’on peut atteindre quand le schéma d’entrée est connu à l’avance. Pour chaque *feature* du modèle chargé, la vue génère automatiquement le *widget* approprié — `ttk.Combobox` pour une variable catégorielle (les choix sont issus du `feature_schema` persisté dans le `.pkl`), `ttk.Spinbox` ou `tk.Scale` pour une variable numérique continue. L’analyse *what-if* utilise des `tk.Scale` dont le `command=` déclenche un recalcul instantané de la prédiction et un re-rendu du graphique SHAP via `matplotlib.figure.FigureCanvasTkAgg` embarqué dans la hiérarchie Tk. Le *batch* CSV passe par `tkinter.filedialog.askopenfilename`, validé côté `services/`.

2

Architecture et ingénierie backend

2.1 Vue d'ensemble en trois couches

L'architecture suit un modèle en couches concentriques où chaque anneau ne dépend que de l'anneau immédiatement intérieur. Cette discipline — héritée de la *Clean Architecture* — explique la résilience du projet aux évolutions successives.

*[Emplacement réservé — diagramme conceptuel
de l'architecture en trois couches (ui / services / core)
à générer ultérieurement via DALL-E 3 / Midjourney.]*

Figure 2.1 – Vue conceptuelle de l'architecture en couches (placeholder).

Couche	Responsabilité	Dépendances autorisées
ui/	Tkinter / ttk, vues, composants, thèmes <i>light/dark</i> , i18n FR/EN.	→ services/
services/	Orchestration métier, agent IA, <i>pipeline service</i> , clients LLM, i18n service.	→ core/
core/	Logique ML pure : <i>loader</i> , préprocessing, entraînement, évaluation, XAI, <i>fairness</i> .	→ config/, utils/

Bénéfice direct de cette discipline

Aucun code Tkinter ne fuit vers `core/`. Réciproquement, aucune référence `scikit-learn` ne contamine la couche UI. Les services agissent comme *façade* d'orchestration et sont donc le **seul** point d'entrée de l'agent IA, qui ne peut pas « tricher » en s'adressant directement aux modèles.

2.2 Agent (Copilot) : raisonnement et flux de décision

2.2.1 Architecture conceptuelle

L'agent IA est matérialisé par le module `services/agent_service.py`. Il fonctionne en boucle agentique bornée (`MAX_TOOL_ITERATIONS = 8`) afin d'éviter tout cycle infini.



Figure 2.2 – Architecture conceptuelle de l'agent — position du Copilot entre l'UI Tkinter, le ToolExecutor, le PipelineService et les providers LLM externes.

Le schéma ci-dessus met en évidence quatre constats d'ingénierie majeurs :

1. **L'agent n'est jamais un raccourci** — même en mode « Agent », il invoque PipelineService comme l'aurait fait un utilisateur cliquant dans l'UI.
2. **Le ToolExecutor est l'unique point de dispatch** — il valide les arguments, capture les exceptions et standardise les réponses. C'est également là que l'on injecte un *rate-limiter* ou un journal d'audit si besoin.
3. **Les providers (Groq, Gemini) sont interchangeables** — le contrat est réduit à une méthode `chat(messages, tools)` qui retourne une structure unifiée.
4. **L'UI ne sait rien des LLM** — le AgentPanel (un `tk.Frame` avec une zone de chat `tk.Text` à tags de couleurs et un `ttk.Entry` de saisie) ne communique qu'avec AgentService, jamais directement avec les API.

2.2.2 Trois modes d'opération

Ask Réponses purement conversationnelles, sans appel d'outil. Le *system prompt* interdit explicitement le *tool-calling*. Cas d'usage : aide pédagogique, FAQ XAI.

Agent

Mode autonome avec *tool-calling* activé. L'agent peut lire l'état du *pipeline*, exécuter des actions et naviguer entre vues. La directive critique « *Visual Pipeline Construction* » le pousse à privilégier la construction visuelle du *pipeline* plutôt qu'un appel monolithique.

Plan

Mode réflexion : l'agent produit un plan structuré *avant* exécution. Adapté aux *workflows* complexes où l'utilisateur veut valider la démarche avant de l'appliquer.

2.3 Tools et flux d'exécution

2.3.1 Orchestration technique

Le contrat de *tool-calling* suit le schéma OpenAI standard, ce qui garantit la portabilité vers tout *provider* compatible. Le `ToolExecutor` désigne la couche qui transforme l'intention de l'agent en action concrète.



Figure 2.3 – Flux de *tool-calling* — boucle agentique complète, de la réception du message utilisateur à la réponse finale, en passant par la détection des *tool_calls* et leur exécution.

Lecture du schéma. La boucle procède en cinq étapes strictement ordonnées :

1. **Build context** — `AgentService` compile l'état courant (statut du *pipeline*, langue de l'utilisateur, mode actif) puis le préfixe au message.
2. **LLM call** — le *provider* reçoit les messages plus le catalogue d'outils.
3. **Detect tool_calls** — si la réponse contient des appels, ils sont parseés ; sinon, le *flow* court-circuite vers l'étape 5.
4. **Execute tools** — le `ToolExecutor` invoque les méthodes correspondantes de `PipelineService`, capture les erreurs et réinjecte les résultats sous forme de messages `tool` dans l'historique.
5. **Return result** — une fois la boucle satisfaite (ou la limite de 8 itérations atteinte), la réponse finale est envoyée à l'UI, qui la rend dans son `tk.Text` avec mise en forme markdown maison.

2.3.2 Catalogue d'outils

Le registre `services/agent_tools.py` déclare quatorze outils, chacun *wrapper* fin autour d'une méthode `PipelineService` existante. Cette discipline garantit qu'il n'existe aucune logique ML exposée uniquement à l'agent.

Outil	Rôle
<code>get_pipeline_status</code>	État global (données, cible, préprocessing, modèles, évaluation).
<code>get_data_summary</code>	Forme, types, valeurs manquantes, statistiques descriptives.
<code>get_columns</code>	Liste exhaustive des colonnes du dataset.
<code>set_target_columns</code>	Définition de la (des) cible(s) supervisée(s).
<code>get_preprocessing_recommendatic</code>	Suggestions fondées sur l'analyse du dataset.
<code>get_preprocessing_issues</code>	Détection des problèmes bloquants pré-entraînement.
<code>get_preprocessing_catalog</code>	Liste exhaustive des étapes de préprocessing disponibles.
<code>add_preprocessing_step</code>	Ajout dynamique d'une étape au <i>pipeline</i> visuel.
<code>run_preprocessing</code>	Déclenchement de l'exécution du <i>pipeline</i> .
<code>get_available_models</code>	Liste des modèles candidats selon la tâche détectée.
<code>run_training</code>	Entraînement multi-modèles asynchrone.
<code>run_evaluation</code>	Calcul des métriques sur le jeu de test.
<code>get_comparison_table</code>	Récupération du tableau comparatif des performances.
<code>navigate_to</code>	Navigation programmée vers une vue Tkinter.

Pourquoi le *wrapper* plutôt que l'accès direct ?

Chaque outil est une coquille fine. Avantages : (i) le même code métier est exécuté qu'on vienne de l'agent ou de l'UI ; (ii) les invariants sont testés une seule fois côté `PipelineService` ; (iii) un ajout futur (audit, télémétrie, *rate-limit*) se branche en un seul endroit.

2.4 Mémoire : contexte et persistance

La mémoire conversationnelle se décline sur trois plans, chacun conçu pour résister à des défaillances différentes :

Mémoire de session (volatile)

L'historique structuré (`user`, `assistant`, `tool`) vit en mémoire vive au sein de `AgentService`. Il est servi tel quel au *provider* lors de chaque tour, sans tronquage tant que la fenêtre de contexte le permet.

Persistance disque (JSON)

Chaque conversation est sérialisée dans `data/chats/` sous forme JSON. Cela autorise la reprise de session après un redémarrage et constitue la base d'un futur historique consultable.

Reprise inline d'un prompt

Un bouton « Edit » sur chaque message utilisateur le recharge dans la `ttk.Entry` d'envoi (via un `bind('<Button-1>', ...)` sur la *tag* correspondante du `tk.Text`). Petite touche UX qui évite la recopie manuelle.

2.5 Providers IA : Groq et Gemini

2.5.1 Un client unifié sans dépendance externe

Le module `services/llm_client.py` expose une interface unique pour les deux *providers*. Le choix radical : n'utiliser que `urllib` de la bibliothèque standard. Cela élimine toute dépendance `pip` pour la couche LLM, allège les installations et neutralise les risques de *breaking changes* côté SDK tiers.



Groq

llama-3.3-70b-versatile



Google Gemini

gemini-pro / gemini-1.5

2.5.2 Intégration Groq — résolution d'un blocage Cloudflare

`GroqProvider` cible l'endpoint OpenAI-compatible `https://api.groq.com/openai/v1/chat/completions`. Durant l'intégration, le service renvoyait systématiquement un `403 Forbidden` dû à un filtre Cloudflare anti-bot. Le diagnostic a révélé que l'absence d'en-tête `User-Agent` suffisait à déclencher

le blocage. La correction — ajout d'un `User-Agent: XAI-Studio/1.0` explicite — a résolu le problème et constitue désormais le comportement par défaut. Une distinction est par ailleurs faite entre `RateLimitError` et `LLMError` générique pour permettre un *back-off* adapté dans `AgentService`.

2.5.3 Intégration Gemini — réparation du function-calling

`GeminiProvider` consomme l'API Google *Generative AI*. La dérive technique observée était subtile : Gemini renvoyait des erreurs HTTP 400 pendant l'exécution du *tool-calling*. Le format attendu côté Google était un *Protobuf Struct* natif, là où nous transmettions une chaîne JSON « stringifiée ». La couche d'adaptation a donc été modifiée pour appeler `json.loads` systématiquement sur les arguments avant transmission, restaurant ainsi la conformité.

2.5.4 Configuration locale et sécurité

Les clés API et le *provider* actif sont persistés dans `config/agent.json`. Ce fichier est explicitement ignoré par `.gitignore` : aucune clé n'a fui dans l'historique git. Côté UI, une `tk.Toplevel` dédiée (`AgentSettingsDialog`) permet de basculer entre *providers*, de valider la clé en ligne (appel HTTP de *ping*) et de récupérer dynamiquement la liste des modèles disponibles.

3

Évolution du projet et discipline d'ingénierie

L'analyse de l'historique `git` permet de relire le projet sous l'angle de la *discipline d'ingénierie* : chaque jalon correspond à une livraison fonctionnelle qui respecte les contrats d'architecture posés au départ. Cette section synthétise l'évolution en cinq grandes phases.

3.1 Phase 1 — Fondations architecturales

Mise en place de la structure modulaire

c9c73cb – *22 fichiers répartis sur 6 couches architecturales*. Dès le premier *commit*, le projet adopte une organisation en couches séparées. Cette décision fondatrice explique la totalité des choix ultérieurs : aucune régression d'architecture n'a été observée par la suite.

Pull request inter-branches

0084e52 / 90377a3 – *ph2-firstpush* mergé depuis la branche *Fatima-Ezzahrae__AKEBLI*. La mise en place immédiate d'un *workflow* de *feature branches* + *pull request* témoigne d'une rigueur de gestion de version inhabituelle à ce stade d'un projet étudiant.

3.2 Phase 2 — Maturation UI et premier élan ML

Préprocessing et rendu HiDPI

a46ee0b – *Improve preprocessing UI and DPI rendering*. Introduction de la détection automatique de DPI et du *scaling* Tk. Sans cette correction précoce, les écrans 4K rendaient une UI floue.

Training Studio, AutoML et NN Builder

3cb7fb3 – *Add training progress ETA, basic AutoML, and NN Builder MVP.* Le **TrainingView** hérite du suivi temps réel, et un *prototype* d'AutoML apparaît. Le *NN Builder* amorce une approche « programmation visuelle » qui préfigure le *pipeline* visuel manipulable par l'agent.

Harmonisation de la navigation

a157e91 – *Update UI flow navigation and preprocessing layout.* Refonte du **Sidebar** et réorganisation des **ttk.Frame** pour homogénéiser la navigation — une étape critique pour la cohérence ergonomique.

Revamp du Training Studio

d337aec – *Revamp training studio UI and NN builder.* Seconde itération majeure du studio d'entraînement : gestion plus fine des hyper-paramètres, amélioration de la composition visuelle.

Modules d'évaluation et XAI

df5a492 – *Add evaluation and XAI dashboard modules.* Apparition du tableau de bord XAI (SHAP, LIME, PDP) qui constitue la *signature* du projet et fonde son identité *Explainable AI*.

Simplification du Training et navigation Eval

a8df019 – *Simplify training view and add evaluation nav.* Mouvement de simplification courageux : le *Training Studio* se concentre sur l'essentiel et cède les fonctions secondaires à l'évaluation.

3.3 Phase 3 — Industrialisation du module de prédiction

Prédiction « schema-aware »

e0520ce – *Prediction module with categorical input support and schema-aware transformations.* Jalon stratégique : le nouveau module de prédiction utilise les métadonnées sérialisées dans le modèle (**input_feature_names**, **categorical_columns**, **feature_schema**) pour rendre le bon *widget* Tk à la volée (**ttk.Combobox** pour les catégories, **tk.Scale** pour les numériques). La transformation appliquée en inférence est strictement la même que celle de l'entraînement — **c'est la garantie silencieuse** qui prévient les bugs les plus fréquents en production ML.

3.4 Phase 4 — L'agent IA comme couche d'orchestration

Intégration du Copilote

fb7a9d9 – *integrate persistent AI Copilot agent with dynamic tool-calling*. Le *commit body* se lit comme une feuille de route d'ingénierie : panneau persistant, client unifié Groq + Gemini sans dépendance externe, validation d'API en ligne, résolution du *403 Forbidden* Cloudflare, réparation du *function calling* Gemini, outils de manipulation dynamique du *pipeline*, bouton « Edit » sur les prompts, retrait strict des *emojis* pour un rendu professionnel, et stockage local *git-ignored* des clés API. **Quatre bugs externes débusqués et corrigés** en un seul jalon — une masterclass de *integration engineering*.

Internationalisation complète FR / EN

1681540 – *Full English / French localization across all UI views*. Déploiement d'un *I18nService* centralisé. Toutes les chaînes des vues sont désormais résolues dynamiquement, ce qui suppose une discipline strict : *zero hard-coded string*. La préférence est persistée dans *config/lang.json*.

3.5 Phase 5 — Consolidation finale

Pipeline dynamique piloté par l'agent

2f987de – *enable agent to add steps directly to dynamic pipeline*. L'agent acquiert la capacité d'ajouter directement des étapes au *pipeline* visuel. C'est la jonction finale entre raisonnement LLM et UI Tkinter — et la démonstration que notre architecture supportait ce pas supplémentaire sans refactoring lourd.

Robustification multi-cibles

f768882 – *stabilize multi-target training, evaluation, and dynamic splits*. Hardening du *pipeline* : prise en charge de plusieurs cibles, *splits* dynamiques, fiabilisation de l'évaluation.

Mise à jour documentaire finale

461e9f2 – *Updated README with latest modifications*. Mise en cohérence du *README* avec l'état livré. Petit *commit*, mais signal clé de la rigueur : la documentation suit le code.

3.6 Synthèse et conclusion

XAI Studio prouve trois propositions à la fois :

1. Tkinter, manipulé avec discipline (`ttk.Style` dynamique, *scaling* HiDPI, conteneurs scrollables sur *Canvas*, *threading* cadencé par `after()`) peut livrer une expérience desktop de qualité professionnelle — et ce, sans framework tiers.
2. Une architecture en couches strictement respectée n'est pas un luxe : elle est ce qui a permis d'intégrer successivement le module XAI, le module de prédiction *schema-aware*, l'agent LLM et l'i18n complète **sans jamais réécrire la couche précédente**.
3. Un agent LLM, lorsqu'il est conçu comme *client d'une API métier* plutôt que comme *tunnel direct vers le code*, ne déstabilise pas l'architecture — il l'enrichit.

L'historique git, lu dans son ensemble, raconte une histoire de prise de décisions techniques justes au bon moment : une fondation modulaire dès le premier *commit*, des améliorations incrémentales successivement validées par des *pull requests*, et une dernière phase de stabilisation qui ferme le projet sur un livrable robuste, documenté et internationalisé. La synergie entre l'élégance d'une UI Tkinter poussée dans ses retranchements et la rigueur d'un *backend* orienté services positionne XAI Studio comme un cas d'étude particulièrement convaincant des bonnes pratiques d'ingénierie logicielle appliquées à un produit de *data science* professionnel.